jigsaw-agile-community-rules

3rd Place Solution

Rank	3
Team	Sergio Manuel Papadakis
Author	Sergio Manuel Papadakis
Topic ID	613324
Version	Original

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613324
Retrieved with Kaggle CLI on 2026-10-21.

First of all, I'd like to express my gratitude to Kaggle and Jigsaw ACRC for organizing and hosting this fascinating competition.

I made public my top submission: notebook.

This solution is based on these ideas

- Since many rules were hidden, I relied on Test-Time Training (TTT) to fit models during inference time.
- I built a strong ensemble of models that can outperform every single model and can withstand the score variations between the public and private leaderboards.
- All the models and the final ensemble were validated against the Public LB.
- I fitted as many models as possible during the 12-hour inference window.
- The final ensemble was composed of 2 sub-ensembles:

1. slow ensemble: composed of LLMs: 2x `Qwen2.5-instruct-14b`, 2x `Qwen2.5-instruct-7b`, 2x `Qwen3-instruct-14b` and 2x `Qwen3-instruct-8b`.
2. fast ensemble: composed of faster models: 2x `bge-base-v1.5`, `Qwen3-embedding-0.6b` and `DeBERTa-v3-small`.

Training Methods

- To perform the pretraining and the TTT a new dataset was built using all the rules, bodies/examples and their corresponding known target.
- The column `subreddit` was discarded. It proved to be unhelpful during both the pretraining and TTT phases.
- The pretraining phase was carried out using this notebook.

Slow ensemble

- The LLMs were pretrained during 1 epoch of training using the public dataset.
- The LLMs pretraining was done in a zero-shot fashion, using the following prompt:
  - System prompt: `"You are given a comment on reddit. Your task is to classify if it violates the given rule. Only respond Yes or No.\nRule: {rule_str}"`
  - User prompt: `"1. ```{example_body_str}```\nViolation:{answer_str}"`
- The outputs of the pretraining phase were LoRA weights. These weights were used later to set the starting point of the TTT phase. This approach helped the models to achieve a better performance during TTT since all the LoRAs already knew the competition's task and the prompt used.
- I used Unsloth lib to train the LLMs during pretraining and TTT. Unsloth gave me the opportunity to place one model per GPU (T4) and to perform training in parallel.
- Inference was performed using vLLM, and the outputs consisted of the embeddings of the last token.

- To fit all the models within the 12-hour inference window, I merged the weights of the two LoRAs for `Qwen2.5-instruct-7b` and ran inference as a single model. The same approach was applied to the two `Qwen3-instruct-7b` models.
- During TTT, I used the final LoRAs to perform inference on both the training and test datasets. The outputs were the embeddings of the final "Violation:" token.
- These embeddings were used to fit a cross-validated batch of classic machine learning (ML) models (`LightGBM`, `XGBoost` and `SVC`). The idea behind this was to use the training dataset (used during TTT) to train these ML models to output a probability that could be ensembled with the corresponding probabilities from the other models.

Fast ensemble

- All the models in the fast ensemble didn't have a pretraining phase and they were only trained during TTT.

bge-base-en-v1.5:

- `bge-base-en-v1.5` model was trained using Triplet Loss as suggested in the public notebook.
- I trained two versions of this model: one using the rules as given (NormalRules) and the other with the rules flipped (FlippedRules) (for example, instead of saying "do not give legal advice," I negated the rule to "give legal advice").
  - NormalRules version: the positive sample in the triplet loss was an example with no rule violation, the negative sample was an example with rule violation, and the anchor was the rule.
  - FlippedRules version: the examples were inverted: the positive sample in the triplet loss was an example with rule violation, the negative sample was an example with no rule violation, and the anchor was the inverted rule.
- After the training, the final model was used to produce embedding for all the examples and the bodies in the dataset. I calculated the positive and negative centroid embeddings and compose the final embedding as: `final_embedding = concat([body_embeddings - positive_examples_centroid_embedding, body_embeddings - negative_examples_centroid_embedding])`
- Using these embeddings a new set of ML models was fitted to predict rule violations. The final predictions were used to feed the fast ensemble.

Qwen3 embedding 0.6b

- I trained a LoRA to predict rule violation using ArcFace loss, the output classes were only two: `rule_violation`, `no_rule_violation` (I also tried other configurations with more classes, for instance, one class per rule, but these didn't work well in my tests).
- As input for this model I used only the body (I also trained to use `'{rule}: {body}'` as input, but it didn't improve the score).
- I used mean pooling to extract embeddings from all bodies and examples in both the training and test datasets.

- Using these embeddings, a new set of ML models was trained to predict rule violations for each rule. The resulting predictions were then used as input for the fast ensemble.

DeBERTa v3 small

- To train the DeBERTa v3 small model I used cross entropy loss and extracted embeddings from the first token (`[CLS]` token).
- As input for this model I used `'{rule}[SEP]{body}'`.
- Using these embeddings a new set of ML models was fitted to predict rule violations.
- The probabilities obtained from the ML model were averaged with the output probabilities of the DeBERTa model. The final predictions were used to feed the fast ensemble.

Partial Ensemble Scores

Model	Public Score	Private Score
2x `Qwen2.5-instruct-14b`	0.90828	0.90248
2x `Qwen3-instruct-14b`	0.92571	0.92049
2x `Qwen2.5-instruct-7b`	0.91024	0.90637
2x `Qwen3-instruct-8b`	0.89598	0.88969
2x `bge-base-v1.5`	0.90952	0.89714
`Qwen3-embedding-0.6b`	0.88856	0.88021
`DeBERTa-v3-small`	0.89546	0.88946

Supplemental Q&A

> **Shivam A. · 2025-10-26T04:42:04.540000**
>
> Congratulations!! Would you also have individual performance of these models on leaderboard ?

> **ShadowCoder · 2025-10-27T07:49:34.167000**
>
> Thanks for sharing,I want to know why you use the final LoRAs to perform inference on both the training and test datasets, during the TTT?Do you mean that during TTT, instead of running the test set first and then using the training set + test set, you can directly use the training set + test set instead of the original test set to perform TTT?

> **Sergio Manuel Papadakis · 2025-10-27T23:36:32.293000**
>
> Thanks @shivamreturns!!! I've added the sub-ensembles performance to my summary, I also made public all the pre-trained LoRA weights if you wanna run some tests using my notebook.

> **Sergio Manuel Papadakis · 2025-10-27T23:49:04.077000**

> Thanks @fondrose,
>
> After performing TTT with Unsloth, I needed to run embedding inference using the final model on the TTT training set (which contains examples labeled as either `violation` or `no_violation`), in order to fit all the classic ML models that produce the final prediction (a probability of rule violation).
>
> I also needed to run inference on the bodies of the `test.csv` file to extract embeddings (I refer to this as the proper test dataset), and then evaluate all the ML models fitted in the previous step.